翁恺课 · 第 7 主题

W7 数组运算 · 练习

对齐翁恺《程序设计入门——C语言》第 7 讲 · 参考答案含「思路 + 常见错法」· 请先独立完成再看答案

填空 W7-1

① 线性查找找不到时返回 ____；不能返回 0，因为 a[0] 是____，会误判。二分查找的前提是数组必须 ____。

填空 W7-2

② 选择排序每轮要"挑最____放前面"；记录最小位置的变量初值取 ____；找到后与位置 ____ 交换。共需重复 ____ 轮（n 个元素）。

填空 W7-3

③ 冒泡排序内层循环条件应写 j < ____；二维数组 m[3][3] 行下标范围 ____~____、列下标范围 ____~____；遍历先____层管行、____层管列。

改错 W7-4

④ 下面程序想冒泡升序，但有 1 处**内层边界错误**（可能越界 + 排序不完整）。先自己找，再对照答案。提示：内层每趟都应"少比一个"。

```
/* debug_wk7_401.c — 冒泡排序（找 1 处边界错误）*/
#include <stdio.h>

int main(void)
{
    int a[6] = {5, 2, 8, 1, 9, 3};
    int i, j, tmp;
    for (i = 0; i < 5; i++) {
        for (j = 0; j < 5; j++) {   /* ? */
            if (a[j] > a[j + 1]) {
                tmp = a[j]; a[j] = a[j + 1]; a[j + 1] = tmp;
            }
        }
    }
    for (i = 0; i < 6; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

编程 W7-5

⑤ 封装 `int find(int a[], int n, int target)` 做线性查找，在 {78, 92, 55, 61, 48, 88} 里找 61 并输出位置。示例：找到 61 → 成绩 61 在第 3 个位置。

编程 W7-6

⑥ 封装 `void sort(int a[], int n)` 做**选择排序升序**，main 里传 {9, 4, 7, 1, 8, 3, 6}（n=7）并输出。示例应输出 `1 3 4 6 7 8 9`。

专业场景 W7-7

⑦ 老师要出"成绩名次表"：把 {78, 92, 55, 61, 48, 88} 用**选择排序改成降序**（挑最大放前面），输出从高到低的成绩。示例应输出 成绩从高到低：92 88 78 61 55 48。

概念 W7-8

⑧ 结合讲义图 W7-1：为什么二分查找比线性查找快？它的代价（前提）是什么？如果对无序数组直接用二分会怎样？「选择排序」和「冒泡排序」最终都能排好，思路各是什么？

参考答案

① 返回 `-1`；不能返回 0 因为 `a[0]` 是**合法位置**（下标从 0 开始），返回 0 会把"真的在 a[0]"误判成"没找到"。二分前提是数组必须**有序**。

思路：用"不可能的下标"表示找不到，-1 正好落在合法范围（0~n-1）之外。常见错法：用 0 或 -n 当找不到，都会撞上合法下标。

② 挑最**小**放前面；`min` 初值取 `i`（当前轮起始位置）；找到后与位置 `i` 交换；共需 **n-1** 轮（最后剩一个不用挑）。

思路：选择排序每轮把"最小元素的位置"记到 min，再换到当前 i。常见错法：min 初值写死 0（每轮都和位置 0 比，错）；或循环只到 n-2 漏最后一轮。

③ 冒泡内层 `j < n-1-i`；二维 `m[3][3]` 行 0~2、列 0~2；遍历外层管**行**、内层管**列**。

思路：冒泡每趟末尾一个元素归位，所以内层上界每趟减 i（少比一个）。常见错法：内层写死 `j < n-1` 导致 `a[j+1]` 越界（见改错④）。

④ 错误：内层条件写 `j < 5`，应为 `j < 5 - i`。写 `j < 5` 时，最后一趟 j 会到 4，访问 `a[j+1] = a[5]`（此时已是归位区，再比较一次冗余），且因为每趟都比满 5 次、没随趟数减少，排序"多余且可能越界"。

gcc 诊断口径（`-Wall -Wextra -O2` 实测）：

```
（编译 0 输出——数组越界属未定义行为，gcc 不保证拦；本示例实测运行甚至"碰巧"仍输出 1 2 3 5
```

```
8 9）
```

诊断口径解读：这是**逻辑边界错 + 未定义行为**，不是语法错误。正因 gcc 不报、运行可能碰巧对，排序类代码的错误**只能靠推演边界**发现——写完把 j 在每个拐点手算一遍，别只信"运行结果看起来对"。

⑤ 参考代码（wk7_p5.c，gcc 实跑 0 警告）：

```c
#include <stdio.h>

int find(int a[], int n, int target);

int main(void)
{
    int score[6] = {78, 92, 55, 61, 48, 88};
    int t = 61;
    int pos = find(score, 6, t);
    if (pos >= 0)
        printf("成绩 %d 在第 %d 个位置\n", t, pos);
    else
        printf("成绩 %d 没找到\n", t);
    return 0;
}

int find(int a[], int n, int target)
{
    int i;
    for (i = 0; i < n; i++) {
        if (a[i] == target) return i;
    }
    return -1;
}
```

实测输出：

```
成绩 61 在第 3 个位置
```

思路：找到立刻 `return i`；扫完没找到 `return -1`。常见错法：用 0 当找不到；或把 `a[i] == target` 写成 `a[i] = target`（赋值，恒真/恒假）。

⑥ 参考代码（wk7_p6.c，gcc 实跑 0 警告）：

```c
#include <stdio.h>

void sort(int a[], int n);

int main(void)
{
```

```
    int a[7] = {9, 4, 7, 1, 8, 3, 6};
    int i;
    sort(a, 7);
    for (i = 0; i < 7; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}

void sort(int a[], int n)
{
    int i, j, min, tmp;
    for (i = 0; i < n - 1; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (a[j] < a[min]) min = j;
        }
        tmp = a[i]; a[i] = a[min]; a[min] = tmp;
    }
}
```

实测输出：

```
1 3 4 6 7 8 9
```

思路：外循环 i 从 0 到 n-2，内循环在 i 之后找最小位置，交换到 i。常见错法：min 忘在每轮重置为 i（沿用上一轮）；或交换写错（`a[i]=a[min]` 覆盖了要换走的值，先存 tmp）。

⑦ 参考代码（wk7_p7.c，gcc 实跑 0 警告）：

```
#include <stdio.h>

int main(void)
{
    int score[6] = {78, 92, 55, 61, 48, 88};
    int i, j, max, tmp;
    for (i = 0; i < 5; i++) {
        max = i;
        for (j = i + 1; j < 6; j++) {
            if (score[j] > score[max]) max = j;  /* 降序：挑最大 */
        }
        tmp = score[i]; score[i] = score[max]; score[max] = tmp;
    }
    printf("成绩从高到低：");
    for (i = 0; i < 6; i++) printf("%d ", score[i]);
    printf("\n");
    return 0;
}
```

实测输出：

```
成绩从高到低：92 88 78 61 55 48
```

思路：把选择排序的"挑最小"改成"挑最大"（`a[j] > a[max]`）就是降序。常见错法：忘了改名（变量叫 min 但逻辑是找最大，可读性差易错）；或比较符号仍写 `<`（又变升序）。

⑧ 二分查找每次把区间砍一半（O(log n)），10 亿个数约 30 次就能找到；线性要扫一遍（O(n)）。它的**代价/前提**是数组必须**有序**。对无序数组直接二分会得出**错误结果**（因为折半的前提"左边都小、右边都大"不成立）。选择排序：**每轮挑最小的放到前面**（锁定位置 i）；冒泡排序：**相邻两两比较，大的往后冒**（每趟把最大沉到末尾）。两者最终都升序，思路不同、都 $O(n^2)$。

思路：一句话记——二分"快但有门槛（有序）"、线性"没门槛但慢"；选排"挑最小放前"、冒泡"大个冒上去"。常见错法：二分忘了先排序，或把 mid 更新写成 `lo=mid` / `hi=mid`（死循环，应 ±1）。

本练习册为自写原创变式题（对齐翁恺《程序设计入门——C语言》第 7 讲），仅作课堂伴读；改错题与编程题参考代码均用 gcc -Wall -Wextra 实测（MinGW-W64 gcc 16.1.0），输出为真实运行结果（含 gcc 警告原文）。原视频/教材版权归原作者所有。